JOBNEST

ONLINE JOB PORTAL

Project README

A web-based job portal designed to connect job seekers with employment opportunities through a simple interface and a C++ backend API.

Project	JOBNEST - Online Job Portal
Backend	C++ (Winsock HTTP server)
Frontend	HTML, CSS and JavaScript
Data storage	Text files (for example, users.txt and applications.txt)

Replace or extend any feature descriptions below to match the final version you publish on GitHub.

1. Project Overview

JOBNEST is an Online Job Portal project intended to provide a common platform for job seekers and administrators. Job seekers can create an account, log in, browse available jobs, save jobs, submit applications and view application-related notifications. The administrator area provides a separate login entry point for administrative access.

The frontend communicates with a locally running C++ backend through HTTP API endpoints. The backend processes requests and stores basic project data in text files.

2. Main Features

- **User registration:** Create an account using details such as name, email, phone number, qualification and password.
- **User login:** Validate registered email and password through the C++ API.
- **Dashboard:** Redirect authenticated users to the dashboard after successful login.
- **Job listing:** Retrieve available jobs from the backend.
- **Job application:** Submit an application for a selected job and prevent duplicate applications for the same user and job.
- **Saved jobs:** Store job IDs saved by a user.
- **Notifications:** Retrieve notifications associated with a user.
- **Application history:** View applications submitted by a user.
- **Admin login:** Separate admin login interface (availability and permissions depend on the final configured admin implementation).

3. Technology Stack

Layer	Technology / Purpose
Frontend	HTML for page structure; CSS for styling; JavaScript for form handling and API requests.
Backend	C++ with Winsock sockets to run a lightweight HTTP API server on Windows.
Communication	HTTP requests using JSON payloads and JSON responses.
Storage	Local text files such as users.txt, jobs.txt, applications.txt, notifications.txt and savedjobs.txt.
Development environment	Windows with a C++ compiler and a modern web browser.

4. Project Folder Structure

A suggested structure based on the files discussed during development is shown below. Your actual folder names may differ; update this tree to reflect the repository you upload.

```
JOBNEST/
   Backend/
     backend.cpp
     users.txt
     jobs.txt
     applications.txt
     notifications.txt
     savedjobs.txt
```

```
Frontend/
  index.html
  login.html
  signup.html
  dashboard.html
  admin-login.html
  css/
  js/
```

5. Requirements

- Windows operating system (the supplied backend uses the Winsock API).
- A C++ compiler that supports Winsock, such as MinGW-w64 or Visual Studio.
- A modern web browser.
- The frontend project files (HTML, CSS and JavaScript).

The backend source includes Windows networking headers such as winsock2.h and ws2tcpip.h. When compiling with a MinGW-based compiler, link against the Winsock library using -lws2_32.

6. Build and Run the Backend

Step 1 — Open the backend folder

Open a terminal in the folder containing backend.cpp. Keep the text data files in the working directory used when the backend executable is launched.

Step 2 — Compile

```
g++ backend.cpp -o backend.exe -lws2_32
```

If you use Visual Studio, create a C++ console project, add backend.cpp and build it with the Windows SDK available.

Step 3 — Start the server

```
backend.exe
```

A successful start should display a message similar to: JOBNEST C++ API running at http://127.0.0.1:8080. Keep this terminal window open while using the website.

Step 4 — Open the frontend

Open the frontend entry page in your browser, or use the local development server recommended by your editor. Ensure the JavaScript API base URL matches the backend address: http://127.0.0.1:8080/api.

7. API Endpoints

Method	Endpoint	Purpose
GET	/api/health	Check whether the backend is running.
GET	/api/jobs	Retrieve the job list.
POST	/api/register	Register a new user.
POST	/api/login	Authenticate a user.
POST	/api/apply	Submit a job application.
POST	/api/applications	Retrieve applications for a user.
POST	/api/saved-jobs	Save a job for a user.
POST	/api/notifications	Retrieve user notifications.

8. Data Files

The backend uses pipe-separated text records. Keep these files in the backend's working directory and do not edit them while the server is writing to them.

- **users.txt:** name, email, password, phone, qualification and skills.
- **jobs.txt:** job ID, company name, job title, salary, skills and category.
- **applications.txt:** applicant name, company name, job title and application status.
- **notifications.txt:** user name followed by a notification message.
- **savedjobs.txt:** user name and saved job ID.

9. Troubleshooting

- **Backend will not start / port already in use:** Close any previous backend process using port 8080, then restart the server.
- **Cannot connect to backend:** Confirm backend.exe is running and the API_URL in the frontend JavaScript is http://127.0.0.1:8080/api.
- **Login fails:** Confirm the account exists in users.txt and that the email and password match the saved record.
- **Jobs do not appear:** Confirm jobs.txt exists, contains valid pipe-separated records and is in the backend working directory.
- **Changes do not appear:** Refresh the browser and check the browser developer console for JavaScript errors.

10. Security and Project Limitations

This implementation is suitable as an academic demonstration, not as a production job portal. Passwords are stored as plain text in the current text-file approach. A production version should use salted password hashing, a database, HTTPS, input validation, authorization checks for every admin operation, secure session management, rate limiting and robust HTTP request parsing.

The backend binds to 127.0.0.1, so it is intended for local use on the same computer. The current lightweight server and text-file storage are not designed for high traffic or concurrent multi-user deployment.

11. Future Enhancements

- Replace text-file storage with MySQL, PostgreSQL or another database.
- Add secure password hashing and token/session-based authentication.
- Implement role-based authorization for job seekers and administrators.
- Add admin tools for creating, editing and deleting job listings.
- Add job search, filters, pagination, profile editing and application status updates.
- Deploy the frontend and backend using a secure hosting environment.

12. Contributing

For an academic or personal GitHub repository, contributions can be made by creating a fork, working on a separate branch, testing the changes locally and submitting a pull request. Document any new API endpoint or data-file format in this README.

13. License

No license has been specified in this document. If you want others to reuse or modify the project, add an appropriate open-source license file (for example, MIT) to the repository after confirming that it matches your intentions.

14. Author

Project: JOBNEST - Online Job Portal

Add your preferred author name, course, institution, contact details and repository URL here before publishing the README.